Fusion: Allocating Intelligence Across Heterogeneous Models

A New Scaling Dimension for AI Systems

Shunfan Zhou
zhousf@fumolab.ai

Jingqi Long
longjq@fumolab.ai

Yiling Zhang
zhangyl@fumolab.ai

Zhiheng Du
william@fumolab.ai

Fumo Lab

Rev. 0.1 — August 2026

EXECUTIVE SUMMARY

AI systems are moving from answering isolated requests to carrying work through evolving trajectories. For a bounded request, the required capability can often be estimated at the outset and judged by one response. In a trajectory, every observation and action changes the task state—and can change what expertise, evidence, or judgment is required next. The demand for intelligence is revealed through execution, not fully specified before it.

At the same time, intelligence supply is becoming a heterogeneous portfolio. Models differ not only in overall strength and cost, but in specialization, failure mode, reliability, and the roles in which they are useful. The best answerer is not always the best explorer, critic, or verifier.

Together, these changes move the system bottleneck. The question is no longer only which model should answer a request, but what intelligence the current state needs next, which source can provide it, and when the evidence is sufficient to act. Routing remains useful for bounded requests; it cannot by itself govern a workflow whose needs become visible only as the work unfolds.

Fumo Lab is building Fusion to solve this *intelligence allocation problem*. Fusion maintains one task state, identifies the decision-relevant uncertainty, and allocates the model–role or tool computation best suited to resolve it. Outputs become attributed evidence, while one authoritative path preserves coherent answers and actions as the loop repeats.

This makes efficiency a consequence of system understanding. Fusion can concentrate frontier judgment where it is load-bearing, use complementary capabilities elsewhere, and stop when further inference has little value. The goal is a performance–compute Pareto improvement that persists from bounded problems to long-running workflows.

Fusion has two sources of progress. Advances in models expand the intelligence available to the system; experience improves how Fusion understands tasks and allocates that supply. Each trajectory provides feedback for a bounded, evaluated, and reversible learning loop. Together, model progress and allocation progress create a new scaling dimension: system capability that can compound faster than frontier-compute consumption.

1 The Unit of Intelligence Is Changing

The dominant language-model interface treats the basic unit of intelligence as a response. A request arrives, a model answers, and the interaction ends. That frame shaped how systems selected models, measured capability, and spent compute.

Agentic systems change that unit of work. They do not merely produce a longer answer; they carry a task through a sequence of observations, decisions, actions, failures, and verifications. The central systems question therefore changes as well: not only whether a model can answer, but whether intelligence can remain effective as the work itself evolves.

1.1 From responses to trajectories

In a bounded request, capability can often be approximated by the quality of one response. The task is visible at the outset, the model acts once, and the answer can be evaluated as a single object.

In an agentic workflow, every transition changes the state. A search result can eliminate the original uncertainty. A failed action can expose a deeper constraint. A user clarification can invalidate a plan. A testable artifact can make verification more valuable than further ideation. What the system should do next depends on what has already happened.

Capability must therefore persist across a trajectory. A strong answer at one turn is not enough if later steps repeat work, lose valid evidence, act from incompatible assumptions, or declare success before the outcome is verified. The system must preserve intent and accumulated evidence while changing how it applies intelligence from one state to the next.

1.2 Demand is revealed through action

The changing requirement is not captured by a single difficulty score. One state may need broad exploration, another specialist knowledge, another precise construction, another adversarial critique, and another a deterministic observation from the environment. A simple but consequential action may require stronger judgment than a difficult but reversible analysis.

Nor can these requirements be fully known before execution. Determining what a task needs can require acquiring the very evidence that the next computation is meant to produce. The apparent hard part may disappear after one tool result; the load-bearing uncertainty may become visible only after a candidate fails.

Intelligence demand in an agentic workflow is therefore *state-dependent*, *multidimensional*, and *partially observable*. It is progressively revealed by doing the work.

1.3 From a model hierarchy to a capability portfolio

While demand is becoming dynamic, the available supply of intelligence is becoming more diverse. Models differ because of training data, scale, architecture, post-training, tool exposure, domain specialization, and deployment surface. These differences produce distinct strengths and failure modes across reasoning, code, science, search, critique, tool use, latency, reliability, privacy, and cost.

The result is not simply a longer leaderboard. Model capability is conditional. The best standalone answerer may not be the most useful critic. A smaller model may contribute more when it searches a different region, supplies specialist knowledge, or performs a bounded check. Two frontier models may be individually strong yet jointly redundant because their errors are correlated.

Cost belongs inside this capability landscape rather than outside it as a commercial filter. It determines how broadly a system can search, how often it can verify, and how many real tasks it can learn from. A capability that can be used selectively and repeatedly has a different system value from one that is available only under an unbounded inference budget.

1.4 Allocation becomes the bottleneck

When the task is bounded and its requirements are visible, model routing is a natural and useful response: characterize the request, choose an appropriate model, and escalate when necessary.

The abstraction breaks down when it is asked to govern an evolving trajectory. A router can classify every turn, but it must still decide what the turn needs before acquiring the evidence that may reveal that need. It usually reduces demand to a class or difficulty level and assumes that one selected model can own the resulting step.

Difficulty is not the real control variable. Knowing that a state is difficult does not reveal whether the system needs another solver, a domain specialist, an external observation, an independent critic, stronger synthesis, or no further computation at all. The load-bearing question is what remains unresolved and what kind of intelligence could resolve it.

The two shifts therefore meet in a new systems problem:

> *Continuously match a partially observed, evolving demand for intelligence with a heterogeneous supply of capability.*

We call this the *intelligence allocation problem*. Its unit of decision is not the model that should own the original request, but the computation that is most valuable at the current state of the work.

2 Fusion: A New Scaling Dimension

Once a task's intelligence requirement can no longer be specified in advance, choosing a model is not enough. The system must discover what the evolving work needs, acquire it from the available capability supply, and repeat that process after the state changes.

Fumo Lab is building Fusion for this transition. Fusion treats intelligence allocation as a persistent system capability: one that operates throughout a trajectory and improves through experience.

2.1 From composition to compound intelligence

Fusion combines a changing portfolio of models with an intelligence layer that learns how to understand the current state, identify what evidence is missing, and decide which capability should be applied next. We call this a *compound intelligence system*.

The distinction between composition and compounding is deliberate. Composition describes what a system contains. Compounding describes how its capability grows. Adding more models creates a composition; learning how their conditional strengths interact with real tasks creates the possibility of compounding.

On a bounded request, allocation can resemble routing optimization. Across a stateful trajectory, it governs how intelligence is assembled, preserved, and redirected at every transition. Across many trajectories, it can learn from the relationship between allocation decisions and outcomes. Allocation therefore becomes a source of system progress in its own right.

2.2 Two coupled loops of progress

Fusion advances through two connected loops. The first operates *within* a trajectory. The current state exposes a decision-relevant uncertainty; Fusion allocates a model–role or tool computation to acquire evidence; the result updates the state; and the system allocates again.

The second operates *across* trajectories. Each completed task reveals how Fusion interpreted the state, which computations it selected, what evidence they produced, which evidence changed the commitment, and whether the outcome succeeded. Those observations improve future task understanding, capability estimation, allocation, and stopping.

Two forms of progress enter these loops:

$$\text{system progress} \longleftarrow \underbrace{\text{model progress}}_{\text{supply}} \times \underbrace{\text{allocation progress}}_{\text{demand}}. \tag{1}$$

This is an architectural claim, not a proposed empirical power law. *Supply-side intelligence improvement* expands what Fusion can draw upon as models advance in reasoning, specialization, reliability, speed, and cost. *Demand-side allocation intelligence* improves how effectively Fusion turns that portfolio into useful evidence and action. Either can advance without waiting for the other; together, their effects can compound.

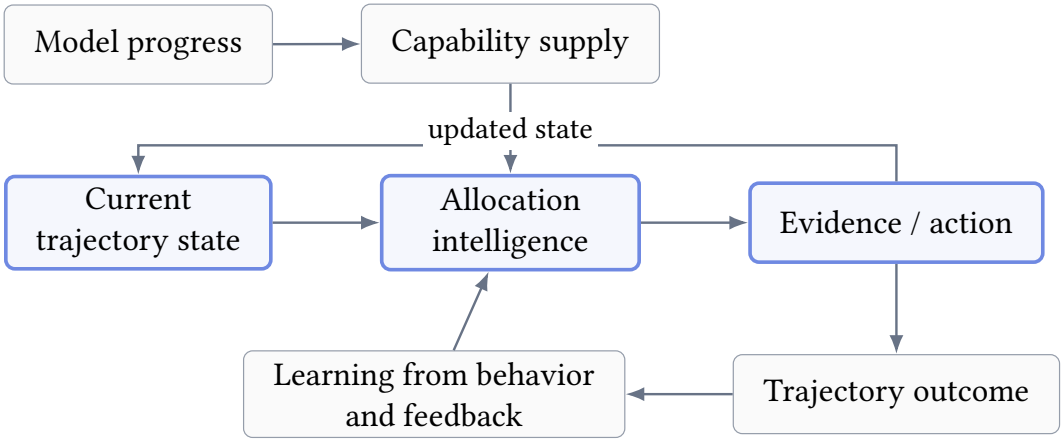

Figure 1: Model progress expands the intelligence available to Fusion. Allocation progress improves the inner trajectory loop and learns from outcomes across trajectories.

The outer loop is recursive in a bounded, system-level sense. Fusion uses traces of its own behavior and outcomes to improve how it assembles intelligence on the next task. It does not imply unrestricted self-modification: policies remain versioned, evaluated, constrained, and reversible. Section 6 describes the learning mechanism and its safeguards.

2.3 Capability beyond proportional compute

The external manifestation of this scaling dimension has two parts. On bounded problems, Fusion should move the performance–compute frontier: comparable quality with less frontier computation, and greater quality at comparable compute. Across agentic trajectories, it should preserve effectiveness as the state changes: higher end-to-end completion, fewer inconsistent transitions, and lower compute per successful task.

This is not a low-cost AI thesis. Cost is one observable consequence of whether the system understands its own work. Redundant frontier calls indicate that it cannot distinguish available evidence from missing evidence. Premature reliance on a weaker source indicates that it cannot recognize where stronger judgment is load-bearing. Allocation intelligence must improve both capability and the productivity of compute.

> *Model progress expands the supply of intelligence. Fusion compounds it through allocation intelligence—within a task and across tasks.*

3 Allocation as Evidence Acquisition

Fusion's governing mechanism follows directly from the intelligence allocation problem. The system does not need to predict the full demand of a trajectory before the work begins. It can discover that demand progressively by treating inference as a closed-loop process of evidence acquisition.

At each point in a workflow, Fusion maintains the current state of the task and asks four questions:

1. What decision-relevant uncertainty remains?
2. What evidence could materially change the next answer or action?
3. Which model–role or tool computation is best suited to acquire it?
4. Is the expected value of that computation greater than its cost, latency, and risk?

The resulting observation updates the task state, and the questions are asked again. Allocation is therefore not a prelude to execution. It is a process that continues throughout execution.

3.1 From task assignment to computation allocation

The distinction changes the unit of orchestration. A router assigns a request to a model. Fusion allocates the *next evidence-producing computation* in the context of an evolving state. Its decision includes not only a model, but a role, a bounded context view, a budget, and a criterion for stopping.

Table 1: Routing and allocation solve different control problems.

	Model routing	Fusion allocation
Unit of decision	Request	Next computation
Demand model	Upfront class or difficulty	Evolving information state
Control action	Choose an answering model	Choose source, role, view, budget, or stop
Output semantics	Answer	Attributed evidence or commitment
Time horizon	One assignment per request or turn	Closed loop within and across the trajectory

This is not a claim that routing is obsolete. Routing remains a useful action inside Fusion when a request is sufficiently bounded. The difference is that it is no longer the governing abstraction for the whole system.

3.2 Decision-relevant uncertainty

Language models expose several signals associated with uncertainty. Token entropy measures variation over expression; semantic entropy can better capture disagreement over meaning [Kuhn et al., 2023, Farquhar et al., 2024]. Fusion must reason at a third level: whether an unresolved variable could change the system's next commitment.

This distinction prevents activity from masquerading as progress. Several models can agree because they share training patterns and blind spots. Many different answers can be irrelevant to the load-bearing decision. Conversely, a single tool observation or counterexample can collapse an entire branch of reasoning.

Fusion therefore values a computation by its expected effect on the decision:

$$\text{value of a call} = \text{expected decision improvement} - \text{compute} - \text{latency} - \text{risk}. \tag{2}$$

The system does not need a perfect Bayesian model of an open-ended task. It needs practical, auditable approximations: unresolved claims, candidate quality, evidence coverage, workflow readiness, failure risk, and the observed conditional value of available sources. Each acquisition can improve these estimates before the next allocation is made.

3.3 Models as conditional evidence sources

Under this view, a model is not a scalar score and a call is not a vote. A model is a conditional source of evidence. Its contribution depends on the task, role, context, evidence already present, provider surface, and budget. The same model may be valuable as a critic and redundant as a second solver. A frontier model may be indispensable for synthesis yet unnecessary for a bounded search or check.

This is why an asymmetric model pool can outperform its apparent ingredients. Fusion is not replacing strong intelligence with weak intelligence. It is separating the different kinds of work hidden inside a model call and sourcing each where it has the greatest marginal value.

3.4 Two principles

The evidence-acquisition view leads to Fusion's foundational design principle:

> *Organize computation around uncertainty, not models.*

But plural evidence creates a second problem. In an agentic workflow, multiple models cannot independently redefine the task, mutate shared state, or issue incompatible actions. Epistemic plurality must not become operational fragmentation. Fusion therefore follows a second principle:

> *One state, many perspectives, one commit.*

Together, the principles define a compound intelligence system. The first determines why computation is acquired. The second determines how multiple computations can contribute without sacrificing coherence and accountability.

4 Designing a Coherent Compound Intelligence System

Fusion's architecture follows from its view of the problem. If intelligence demand changes with the state, the system must preserve that state. If demand is multidimensional, calls must have distinct evidence roles. If model capability is conditional, allocation must be role- and state-aware. If many sources can contribute but only one action can occur, evidence production and commitment must have different authority.

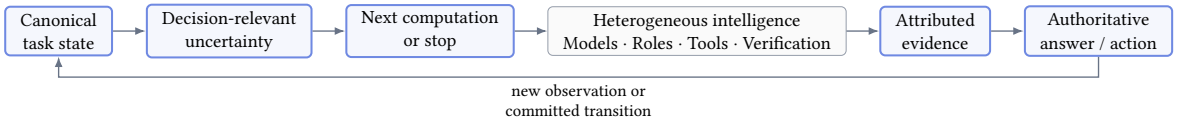

Figure 2: Fusion continuously matches evolving intelligence demand with heterogeneous supply. Every observation can change the next allocation.

4.1 Because demand evolves: one canonical state

Fusion begins with one normalized history of the work: user intent, constraints, prior decisions, tool observations, produced artifacts, failures, and acceptance conditions. This canonical state is the reference against which the system determines what has changed, what is supported, and what remains unresolved.

Different computations receive purpose-built projections of that state. An explorer may need the open question without the full interaction history. A critic may need the candidate and its acceptance criteria. The authoritative synthesizer retains the native conversation and action contract. Bounded views reduce irrelevant context and accidental authority while making every contribution auditable against what its source was allowed to observe.

The state also distinguishes the structure of the overall task from the local progress of the workflow. A difficult umbrella task does not justify maximum computation on every turn. Once an artifact exists, a deterministic test may be more valuable than another proposed solution. After a failed action, the failure evidence should change the allocation rather than reset the task.

4.2 Because demand is multidimensional: role-conditioned evidence

Fusion does not represent intelligence demand with a single difficulty score. It identifies the kind of unresolved evidence the current decision needs. Roles encode that acquisition target. The concrete taxonomy is implementation-dependent and may evolve with the model supply and task distribution.

A role is therefore not a persona used to create stylistic diversity. It specifies what observation the system is trying to obtain. This makes outputs comparable by contribution to the current decision rather than by generic answer quality.

4.3 Because capability is conditional: asymmetric allocation

Given the state and evidence need, Fusion chooses a source, role, context view, and budget. It can vary the number and type of producers, the capability tier of the authoritative synthesizer, and the depth of independent audit. It can also decide that no additional model call is warranted.

These decisions depend on a versioned capability view rather than a permanent ranking. Fusion asks which source has historically contributed novel, valid, and decision-changing evidence under similar

conditions. Cost, latency, reliability, complementarity, and error correlation belong in the same view as reasoning or domain performance.

This naturally produces an asymmetric portfolio. Efficient models can extend the breadth of search and audit where their marginal contribution is high. Specialists can supply narrow capability absent from a general model. Frontier models can concentrate on integration and decisions that require stronger judgment. Asymmetry is not a fixed topology; it is the result of conditional allocation.

4.4 Because evidence is plural: one authoritative commit

Every producer contribution enters Fusion as bounded, attributed evidence. A candidate solution, a criticism, a test result, and a suggested tool call do not have the same semantics. Fusion preserves those differences rather than flattening all outputs into interchangeable answers or votes.

The authoritative synthesizer must judge which claims are load-bearing, reconcile or reject incompatible evidence, and satisfy the original client contract. Producer tool calls remain advisory; only the authoritative path can emit an external action. Model identities may be hidden from synthesis to reduce prestige bias while remaining available in system traces for evaluation and learning.

This authority boundary is a direct response to agentic state. Multiple processes may investigate the world, but they cannot safely fork the world state. Fusion separates broad epistemic search from singular operational responsibility.

4.5 Because information value changes: adaptive budgets and stopping

The marginal value of computation changes as evidence accumulates. Early in a task, several independent paths may be valuable. Later, another candidate may be redundant while a precise verification is decisive. Once the next commitment is stable, further inference adds latency and cost without adding capability.

Fusion therefore treats call admission, fan-out, escalation, and stopping as parts of the same allocation policy. Compute is expanded when unresolved risk justifies it and contracted when the evidence is sufficient. Efficiency is not created by always choosing a cheaper model; it is created by declining low-value computation at every capability tier.

4.6 Because the work persists: continuity across the trajectory

For a one-shot request, commitment produces an answer. For an agentic workflow, it produces one state-consistent transition and then the loop begins again. Fusion preserves the same evidence and authority semantics through four recurring modes:

$$\textsc{Discover} \rightarrow \textsc{Execute} \rightarrow \textsc{Verify} \rightarrow \textsc{Commit}. \tag{3}$$

Discovery seeks the observation that best separates live hypotheses. Execution makes one coherent change. Verification connects recorded evidence to explicit acceptance conditions. Commitment occurs only when material conditions are supported. This continuity is what makes allocation intelligence a property of the full system rather than an optimization around an individual model call.

5 Performance Beyond Proportional Compute

The promise of allocation intelligence must be visible from outside the system. Fusion should not require a customer to choose between frontier-level capability and practical deployment efficiency. It should move the frontier itself.

5.1 A performance–compute frontier across three capability regimes

Our current evaluation spans scientific reasoning, policy-constrained tool use, and agentic coding through GPQA Diamond, τ^3-Banking, and Terminal-Bench v2.1, respectively [Rein et al., 2024, Shi et al., 2026, Merrill et al., 2026]. These workloads expose different failure modes, but ask the same systems question: how much capability can be delivered for the compute consumed? Table 2 reports both score and the cost in US dollars of completing each full benchmark run.

Table 2: Reported score and full benchmark-run cost. Lower cost and higher score are better. Within each comparison cohort, the highest score and lowest cost are bold. Per-run costs are rounded to the nearest dollar.

	GPQA Diamond		τ^3-Banking		Terminal-Bench v2.1	
System	Score	Cost	Score	Cost	Score	Cost
Fusion Max	**94.9%**	**13**	**49.5%**	**145**	86.5%	**74**
GPT 5.6 Sol (max)	94.1%	22	44.3%	251	**88.0%**	86
Claude Fable 5 (fallback)	92.6%	45	38.1%	322	84.6%	299
Sakana Fugu Ultra	94.4%	95	41.2%	224	79.8%	144
Fusion Code	**93.5%**	**7**	**48.5%**	159	**89.9%**	71
Claude Opus 5 (max)	93.2%	18	42.1%	193	89.1%	215
Kimi K3	**93.5%**	30	46.0%	**122**	85.0%	**56**

Figure 3 places the focal comparisons from Table 2 in a wider set of native benchmark coordinates. The additional reference points include models commonly used by customers before August 2026 and representative lower-cost operating points reported by Artificial Analysis [Artificial Analysis, 2026]. Each panel plots native benchmark score against full benchmark-run cost; no benchmark weighting or cross-benchmark normalization is introduced. The Banking and Terminal-Bench panels use linear cost axes. The GPQA panel uses a marked broken linear axis to give its dense $0–40 region more space while preserving actual cost tick values. In every panel, the preferred direction is toward the upper left.

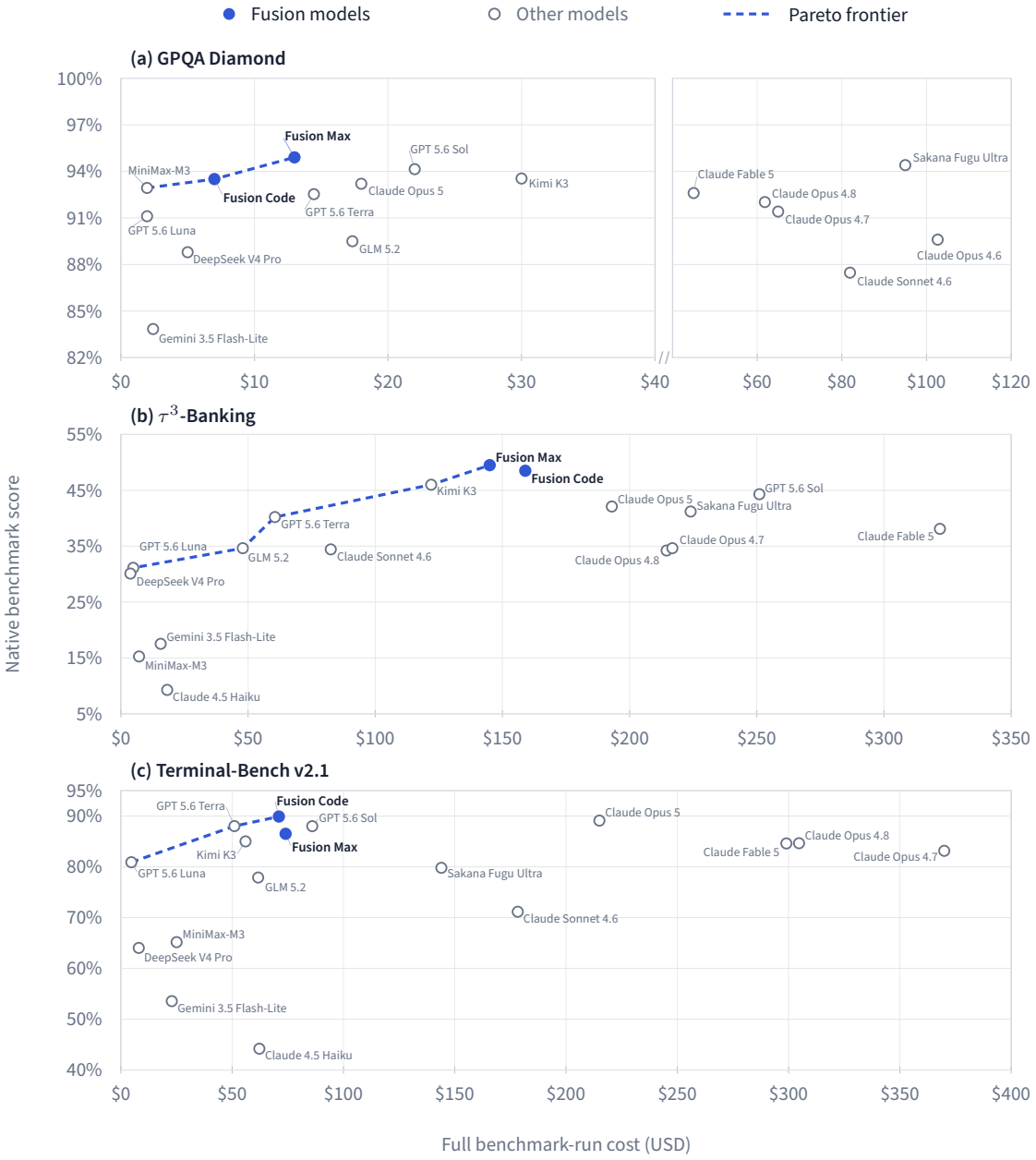

Figure 3: Native score–cost frontiers for the three reported benchmarks. Blue points are Fusion models and hollow gray points are other models. Focal comparison values are reported in Table 2; the additional points use Artificial Analysis scores and full-run cost accounting for models released before August 2026. The GPQA cost axis is broken after $40 to enlarge the dense low-cost region, and its score window begins at 82%; Claude 4.5 Haiku's 67.2% GPQA result is therefore off-scale. A model is otherwise omitted from a panel when no corresponding result is reported. Dashed blue lines connect the observed non-dominated points and are not fitted curves.

Fusion Max reduces the aggregate cost of this fixed three-benchmark portfolio from $359 to $232 relative to GPT 5.6 Sol, a 35.4% reduction. It scores higher on GPQA Diamond and τ^3-Banking and remains within 1.5 percentage points on Terminal-Bench. Against Claude Fable 5, Fusion Max scores higher on all three workloads while reducing each full-run cost by 55–75%; aggregate cost falls from $666 to $232.

Fusion Code presents a second operating point. It meets or exceeds the reported score of Claude Opus 5

on all three workloads while reducing full-run cost by 18–67%; aggregate cost falls from $426 to $237. Kimi K3 illustrates a different trade-off: Fusion Code matches or improves its score on all three workloads, but spends more on the Banking and Terminal-Bench runs. We therefore present that comparison as a capability gain, not as a universal cost reduction.

The result is a Pareto claim, not a discount claim. Fusion retains stronger judgment where it is valuable and removes the assumption that every intermediate unit of reasoning must be purchased at the same capability tier. A system that is only cheaper because it silently accepts worse outcomes has not demonstrated allocation intelligence.

5.2 Why asymmetry changes the economics of capability

The cost profile follows directly from Fusion's architecture. A frontier-heavy orchestration design assembles its worker pool from several of the most capable general-purpose models. This can increase quality, but it also duplicates the most expensive class of computation across planning, generation, critique, and verification. Fusion instead composes a wider capability spectrum. It asks where frontier intelligence is decisive and where a specialized or efficient source can add more information per unit of compute.

Sakana Fugu Ultra provides a useful external operating point [Tang et al., 2026]. It uses learned orchestration across a frontier-heavy worker pool, while Fusion uses task-state understanding to allocate across an asymmetric supply of models. In our available full-suite runs, it reaches 94.4% on GPQA Diamond, 41.2% on τ^3-Banking, and 79.8% on Terminal-Bench, at full-run costs of $94.55, $224.14, and $143.61. Its GPQA result is 185/196 scored cases; two additional selected cases ended in execution errors.

These results do not support a blanket order-of-magnitude cost claim. Across the fixed three-benchmark portfolio, Sakana Fugu Ultra costs $462.30, compared with $232 for Fusion Max: Fusion Max costs about 49.8% less while scoring higher on all three workloads. Costs differ by roughly an order of magnitude on one workload, not across the portfolio. On GPQA, where the two systems are within 0.5 percentage points, Sakana Fugu Ultra costs 7.3 times as much as Fusion Max and 13.5 times as much as Fusion Code. The Banking and Terminal-Bench runs instead show a joint capability and cost advantage for Fusion, rather than cost parity at matched quality.

The comparison is directional rather than perfectly controlled: model and provider snapshots differ, the Banking and Terminal-Bench runs are official-compatible rather than identical submissions, and the Terminal-Bench run is not leaderboard-comparable. Even within that boundary, the architectural contrast remains useful. One system primarily orchestrates frontier intelligence; the other learns where frontier intelligence is necessary.

The point is not that an asymmetric pool is inherently superior. A weak source that adds misleading or redundant evidence is not efficient at any price [Li et al., 2026]. The advantage appears only when the system has the capability understanding and task-state awareness to use asymmetry selectively. That allocation layer—not the mere presence of smaller models—is the source of the frontier shift.

5.3 Effectiveness that survives the workflow

One-shot benchmarks capture only part of Fusion's value. In an agentic system, the demand for intelligence changes after every observation and action. A design that improves a single answer can still fail over a trajectory by repeating work, losing state, acting on incompatible plans, or declaring success before verification.

Fusion is designed so that its benefit persists across the workflow. The same canonical state, uncertainty model, and authority boundary apply whether the system is investigating, editing, testing, recovering

from failure, or committing a result. The relevant outcome becomes successful task completion, not the quality of an isolated response. The relevant cost becomes compute per successful trajectory, not price per API call.

A LONG-HORIZON TRACE: ADAPTATION, NOT REPETITION

In the *path-tracing* task from Fusion Code's full Terminal-Bench v2.1 run, the system had to reconstruct an image produced by an unknown binary as a dependency-free C program whose compressed source had to satisfy a strict size limit. Its initial inspection path failed because basic utilities were absent. It changed methods, using available scripting and binary-analysis tools to inspect the image, analyze the binary, recover the underlying scene, implement a compact renderer, compile it, and compare the generated output against the target.

The trajectory ran for about 29 minutes across 49 agent steps. Before completion, the system measured a normalized L2 similarity of 0.999998; the benchmark's independent verifier then passed all five tests, including compilation, dependency, execution, and similarity checks. This single trace is not an aggregate claim. It illustrates a property that aggregate evaluation must test: when execution invalidates an assumption, the system can preserve state, reduce the new uncertainty, change its next action, and still close the loop with external verification.

We therefore evaluate long-horizon behavior in terms of end-to-end completion, total cost per success, frontier calls per trajectory, state consistency, recovery after failed actions, premature completion, and degradation as workflow length increases. This is where allocation intelligence becomes a system capability rather than a routing optimization.

5.4 A transparent standard of evidence

Public comparisons should expose the full operating point: task set, model and policy versions, scorer, attempts, timeout, concurrency, cache treatment, latency, and all internal model usage. Client-visible tokens are not the total cost of orchestration. Infrastructure and scorer failures must remain separate from system failures. Fumo Lab will report these boundaries explicitly, especially where a fast-moving external system cannot be reproduced under a perfectly matched protocol.

6 The Compounding Advantage

Model progress expands what Fusion can draw upon. Allocation progress determines how much of that supply becomes useful system capability. The latter is where Fusion develops its own compounding advantage. An asymmetric model pool can be copied and a routing graph can be reconstructed; an empirical understanding of how capabilities interact with real tasks—and a system that improves that understanding through use—is harder to reproduce.

6.1 A living map of model capability

Leaderboards describe average model performance on predetermined tasks. Fusion needs a different object: a conditional capability map. It asks how likely a model is to contribute useful evidence for a particular kind of task, under a particular role, at a particular workflow state, given the evidence already available. Query–model interaction models provide one starting point [Song et al., 2025]; Fusion requires a richer, state-dependent map of conditional contribution.

The map includes capability and operating characteristics together: model and provider version, task dimensions, role, context, output validity, latency, cost, observed redundancy, error correlation, syn-

thesizer use, and downstream outcome. It is versioned because models change. It is uncertainty-aware because new models and rare tasks begin with limited evidence. And it measures interaction, because the best standalone answerer is not necessarily the most valuable next contributor.

This systematic evaluation is what makes heterogeneous allocation reliable. Without it, asymmetry is merely an intuition that cheaper models might help. With it, the system can predict where a capability is complementary, where it is redundant, and where the risk requires escalation.

6.2 The allocation data flywheel

Every Fusion task can produce a structured learning record: the state the system observed, the uncertainty it identified, the call it admitted, the role and model it selected, the evidence returned, the decision committed, the resources consumed, and the eventual outcome.

Prior work shows that multi-model interaction logs can support routing, reasoning-pattern retrieval, and capability distillation [Feng et al., 2026]. Fusion extends this learning substrate with task state, evidence use, actions, and downstream outcomes.

These records improve three layers at once:

- **task understanding**: a richer representation of what real users are trying to accomplish and where their workflows become uncertain;
- **capability understanding**: empirical knowledge of what each model contributes under real conditions; and
- **allocation policy**: better admission, role assignment, escalation, synthesis, and stopping decisions.

More use creates a wider view of task demand. Better task understanding creates better allocation. Better allocation creates more informative observations about model capability. The result is a flywheel that is grounded not only in benchmark labels, but in the distribution of real tasks and feedback that the system encounters.

This flywheel is difficult to reproduce from the surface behavior of Fusion. It depends on the joint history of tasks, model versions, allocations, counterfactual probes, and outcomes. The visible architecture is modular; the accumulated model of conditional contribution is proprietary and compounding.

6.3 From designed allocation to learned allocation

Fusion currently combines structured task perception with explicit policy boundaries. This makes allocation interpretable while the learning base grows. The same architecture exposes a stable end-to-end learning interface:

$$\begin{aligned}&(\text{task state}, \text{available evidence}, \text{model supply})\\ &\quad\longrightarrow (\text{next source}, \text{role}, \text{budget}, \text{stop}).\end{aligned} \tag{4}$$

Fumo Lab has begun training models that map user input and evolving task state directly to allocation decisions. Producer behavior supplies model feedback: which sources generated novel, valid, or decision-changing evidence. Synthesizer behavior and downstream outcomes supply system feedback: which evidence was used, which commitment succeeded, and where the workflow failed. Together, they create a path toward reinforcement learning from model and environment feedback for the allocation policy itself.

Learning from production traces requires discipline. The system only observes the outcomes of calls it chose to make, so naive training would reinforce its existing blind spots. Controlled exploration, shadow calls, candidate ablations, offline replay, and local credit assignment are necessary to estimate what would have happened under another allocation. Privacy, provenance, evaluation, and rollback must be properties of the learning loop, not afterthoughts.

6.4 Bounded system-level recursive improvement

Fusion improves through two coupled streams. New and better models expand the supply of intelligence. Better task understanding and allocation extract more value from that supply. Improvements in either stream change the data available to the other: stronger producers reveal new possibilities, while a stronger allocator discovers where each producer matters.

This is recursive improvement in a bounded, system-level sense. Fusion uses observations of its own decisions and outcomes to improve how future intelligence is allocated. It does not imply unrestricted self-modification or an unobservable loop. Policies remain versioned, evaluated, constrained, and reversible.

> *Fusion improves when models improve—and improves again when it learns how to use them.*

7 A Broader Shift in AI Systems

Fusion is part of a broader movement from monolithic model invocation toward systems that actively shape inference. Routing and cascade systems demonstrate that quality and cost can be optimized jointly [Chen et al., 2023, Ong et al., 2024, Aggarwal et al., 2024]. Ensembling and Mixture-of-Agents systems show that multiple candidates can improve a final answer when their contributions are complementary [Jiang et al., 2023, Wang et al., 2024, Lu et al., 2026]. Learned orchestrators increasingly treat worker selection, roles, and topology as policy decisions [Zhang et al., 2025, Yue et al., 2025, Nielsen et al., 2026, Xu et al., 2026, Tang et al., 2026]. Long-horizon agents extend the problem from requests to evolving trajectories [Zhang et al., 2026, Koh et al., 2026].

Fusion shares the conviction that inference should be adaptive and learnable, but chooses a broader control object. A router asks which model should answer. An ensemble asks which answers should be combined. An orchestrator often asks which worker graph should execute. Fusion asks what decision-relevant evidence the current task needs next, which capability can provide it, and whether the system is ready to commit.

That distinction connects several previously separate concerns. Model choice, role, fan-out, tool use, verification, stopping, cost, and workflow phase all become aspects of one allocation decision. The conceptual roots include active information acquisition and value-of-information reasoning [Houlsby et al., 2011, Golovin and Krause, 2011]; the system contribution is to make those ideas practical under natural-language ambiguity, correlated model errors, heterogeneous operating costs, and changing external state.

The important transition is not from one model to many models. It is from static consumption of intelligence to adaptive allocation of intelligence.

8 What Fusion Makes Possible

Allocation intelligence changes what an AI system can become. It decouples the system's capability from the profile of any one model and turns a changing model ecosystem from an integration burden into a source of continuous improvement.

8.1 An intelligence layer above models

Applications today are often rebuilt around the context limits, tool behavior, latency, and failure modes of a chosen model. Fusion creates a more stable layer above that volatility. New models can enter as sources of capability; existing models can change roles as their relative strengths evolve; provider and deployment constraints can be incorporated without redefining the task.

This does not make models interchangeable commodities. It makes their differences usable. A breakthrough frontier model strengthens the system's highest-leverage decisions. A domain model adds specialized knowledge. A local model adds privacy or low-latency capability. A verifier adds grounded evidence. Fusion can absorb each advance according to the uncertainty it is suited to resolve.

8.2 Adaptive intelligence for real workflows

As AI systems take on longer tasks, no fixed allocation chosen at the beginning can be assumed to remain optimal. The task changes when code runs, a user clarifies intent, a tool returns unexpected data, or a test falsifies the current plan. Fusion's closed loop allows the composition of intelligence to change with it.

This makes a new class of system possible: one that can explore broadly and then narrow; spend frontier judgment around consequential moments; invoke verification when claims become testable; recover from failures without discarding valid state; and stop when the outcome is sufficiently supported. The system behaves as one reliable intelligence to its caller while adapting its internal composition throughout the trajectory.

8.3 A continuously improving performance frontier

The near-term benefit of Fusion is a better balance of capability and compute. The longer-term potential is a frontier that keeps moving for two independent reasons. The underlying model ecosystem improves, and the allocation layer learns from a growing distribution of tasks and outcomes.

The first source of progress is inherited from the model ecosystem. The second is generated inside the compound system: Fusion observes its own allocations and outcomes, learns which capabilities mattered, and changes how it allocates future intelligence. Their interaction is what turns a model portfolio from a composition into a compounding system.

This creates leverage beyond inference savings. A better allocator can make new model capabilities useful sooner, identify where expensive intelligence is actually load-bearing, and reveal gaps that should guide model training or specialization. The system becomes both a consumer of model progress and an instrument for understanding where further progress would be most valuable.

8.4 Boundaries that preserve trust

The vision also creates obligations. Model confidence is imperfect; uncertainty cannot be reduced to one universal score; creative tasks may have no singular truth; and model sources often share hidden failure modes. Outcome feedback can be delayed or ambiguous, while consequential actions demand a higher standard than benchmark answers.

Fusion's principles are designed for these realities. Evidence remains scoped and attributed. Authority remains singular. Allocation policies can be conservative where the cost of error is high. Learning remains bounded by versioning, evaluation, privacy, and rollback. These constraints do not limit the compound system; they make its improvement legible and deployable.

The objective is not an opaque machine that calls more models. It is an accountable intelligence layer that knows which capabilities to bring to bear, what evidence supports its decisions, and when it has done enough.

9 Conclusion

Fumo Lab's vision is a compound intelligence system whose progress is not limited to the capability curve of a single model. Fusion advances through two coupled forces: the model ecosystem supplies stronger and more diverse intelligence, while the allocation layer learns how to turn that supply into better outcomes.

The need for this system is becoming more pronounced. Agentic work turns a request into a trajectory in which the capability and evidence required next are revealed through execution. At the same time, the model ecosystem is becoming increasingly heterogeneous. Request-level routing can choose an answering model, but it cannot fully determine in advance which uncertainty will become load-bearing as the work unfolds.

Fusion addresses this gap by organizing computation around decision-relevant uncertainty. It treats models and tools as conditional sources of evidence, preserves one canonical state, and reserves answers and actions for one authoritative commitment path. This allows heterogeneous capabilities to function as one coherent intelligence across both one-shot problems and long-running workflows.

The immediate result is the possibility of a Pareto improvement: frontier-level performance with materially lower frontier-compute consumption. The longer-term result is a system that learns from its own operation. Allocation decisions, producer behavior, synthesizer use, and downstream outcomes improve the task representation, capability map, and allocation policy that govern future work.

This is Fusion's bounded form of system-level recursive improvement. Model progress creates more intelligence; allocation progress determines how much of it becomes useful; experience improves the allocation of the next task. Fumo Lab's ambition is to make those forces compound.

References

Pranjal Aggarwal et al. AutoMix: Automatically mixing language models. In *Advances in Neural Information Processing Systems*, 2024.

Artificial Analysis. Independent ai model evaluations. https://artificialanalysis.ai/evaluations, 2026. Accessed 2026-08-24.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630:625–630, 2024.

Tao Feng et al. FusionFactory: Fusing LLM capabilities with multi-LLM log data. *Transactions on Machine Learning Research*, 2026.

Daniel Golovin and Andreas Krause. Adaptive submodularity: Theory and applications in active learning and stochastic optimization. *Journal of Artificial Intelligence Research*, 42:427–486, 2011.

Neil Houlsby, Ferenc Huszár, Zoubin Ghahramani, and Máté Lengyel. Bayesian active learning for classification and preference learning. *arXiv preprint arXiv:1112.5745*, 2011.

Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion. *arXiv preprint arXiv:2306.02561*, 2023.

Jing Yu Koh, Ruslan Salakhutdinov, and Daniel Fried. Multi-agent computer use. *arXiv preprint arXiv:2606.01533*, 2026.

Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *International Conference on Learning Representations*, 2023.

Junyou Li et al. Rethinking Mixture-of-Agents: Is mixing different large language models beneficial? *Transactions on Machine Learning Research*, 2026.

Yuxuan Lu et al. Mixture of complementary agents for robust LLM ensemble. *arXiv preprint arXiv:2605.24048*, 2026.

Mike A. Merrill et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.

Didrik Nielsen et al. Learning to orchestrate agents in natural language with the Conductor. *International Conference on Learning Representations*, 2026.

Isaac Ong et al. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof Q&A benchmark. In *First Conference on Language Modeling*, 2024.

Quan Shi, Alexandra Zytek, Pedram Razavi, Karthik Narasimhan, and Victor Barres. τ-Knowledge: Evaluating conversational agents over unstructured knowledge. *arXiv preprint arXiv:2603.04370*, 2026.

Wei Song, Zhenya Huang, Cheng Cheng, Weibo Gao, Bihan Xu, GuanHao Zhao, Fei Wang, and Runze Wu. IRT-Router: Effective and interpretable multi-LLM routing via item response theory. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025.

Yujin Tang et al. Sakana Fugu technical report. *arXiv preprint arXiv:2606.21228*, 2026.

Junlin Wang et al. Mixture-of-Agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*, 2024.

Yihang Xu et al. TRINITY: An evolved LLM coordinator. *International Conference on Learning Representations*, 2026.

Shengbin Yue et al. MasRouter: Learning to route LLMs for multi-agent systems. *Proceedings of the Association for Computational Linguistics*, 2025.

An Zhang, Kai Feng, and Jiaxuan You. Router-R1: Teaching LLMs multi-round routing and aggregation via reinforcement learning. *Advances in Neural Information Processing Systems*, 2025.

Zeyu Zhang et al. MTRouter: Cost-aware multi-turn LLM routing with history-model joint embeddings. *Proceedings of the Association for Computational Linguistics*, 2026.